

Настроить MCP-сервер rules-mcp на другом ПК. Пошаговая инструкция.

Шаг 1: Исправить `opencode.jsonc` (если ошибка `mcpServers`)

Если при запуске OpenCode ошибка "Unrecognized key: mcpServers" —
в файле `opencode.jsonc` заменить ключ "mcpServers" на "mcp".

Шаг 2: Записать файл `server.py`

Путь: `C:\AIS\AI\rules-mcp\rules_mcp\server.py`

Скопировать весь текст ниже и записать в этот файл (UTF-8 без BOM):

```
--- НАЧАЛО ФАЙЛА server.py ---
```

```
"""
```

```
rules-mcp -- MCP-сервер правил проекта AIS.
```

```
Предоставляет инструменты для поиска багов, кодировок, сокращений и preflight-проверок.
```

```
Запуск: python -m rules_mcp.server
```

```
"""
```

```
from __future__ import annotations
```

```
import os
```

```
import re
```

```
from pathlib import Path
```

```
from fastmcp import FastMCP
```

```
mcp = FastMCP("rules-mcp")
```

```
PROD_ROOT = Path(os.environ.get("AIS_PROD_ROOT", r"C:\AIS\AI\Prod"))
```

```
_changelog_cache: list[str] | None = None
```

```
_abbrev_cache: dict[str, str] | None = None
```

```
def _get_changelog() -> list[str]:
```

```
    global _changelog_cache
```

```
    if _changelog_cache is None:
```

```
        path = PROD_ROOT / "rules" / "changelog.md"
```

```
        if path.exists():
```

```
            text = path.read_text(encoding="utf-8")
```

```
            _changelog_cache = text.splitlines()
```

```
        else:
```

```
            _changelog_cache = []
```

```
return _changelog_cache
```

```
def _get_abbreviations() -> dict[str, str]:
```

```
    global _abbrev_cache
```

```
    if _abbrev_cache is None:
```

```
        path = PROD_ROOT / "rules" / "abbreviations.md"
```

```
        if path.exists():
```

```
            text = path.read_text(encoding="utf-8")
```

```
            _abbrev_cache = _parse_abbreviations(text)
```

```
        else:
```

```
            _abbrev_cache = {}
```

```
    return _abbrev_cache
```

```
_ABBR_HEADER_KEYS = {
```

```
    "сокращение", "имя", "тип", "статус", "утилита", "команда",
```

```
    "файл", "термин", "полное имя / путь", "значение", "как завершить",
```

```
    "копирование", "сохранение в файл", "команда для модели",
```

```
}
```

```
def _parse_abbreviations(text: str) -> dict[str, str]:
```

```
    result: dict[str, str] = {}
```

```
    for line in text.splitlines():
```

```

stripped = line.strip()

if not stripped.startswith("|"):

    continue

cells = [c.strip() for c in stripped.strip("|").split("|")]

if len(cells) < 2:

    continue

key_raw, value = cells[0], cells[1]

if key_raw and set(key_raw) <= set("-: "):

    continue

key_clean = key_raw.replace("**", "").replace("`", "").replace("\\", "").strip()

if not key_clean:

    continue

if key_clean.lower() in _ABBR_HEADER_KEYS:

    continue

for alias in key_clean.split("/"):

    alias = alias.strip()

    if alias:

        result[alias] = value

return result

```

```

def _extract_bug_entries(lines: list[str], keyword: str) -> list[dict[str, str]]:

    kw = keyword.lower()

    entries: list[dict[str, str]] = []

```

```

current: dict[str, str] = {}

for line in lines:

    stripped = line.strip()

    if stripped.startswith("## ") or stripped.startswith("### "):

        if current:

            if kw in current.get("text", "").lower() or kw in current.get("title", "").lower():

                entries.append(current)

            current = {"title": stripped.lstrip("# ").strip(), "text": ""}

        elif current:

            current["text"] += line + "\n"

        if current and (kw in current.get("text", "").lower() or kw in current.get("title", "").lower()):

            entries.append(current)

    return entries[-8:] if len(entries) > 8 else entries

```

```

ENCODING_TABLE = {

    ".ps1": {"encoding": "UTF-8", "bom": "с BOM (обязательно)", "note": "Без BOM - PS 5.1 читает как CP1251 - кракозябры"},

    ".json": {"encoding": "UTF-8", "bom": "без BOM (обязательно)", "note": "С BOM - Bug-044: OpenCode не грузит плагины"},

    ".jsonc": {"encoding": "UTF-8", "bom": "без BOM (обязательно)", "note": "С BOM - Bug-044: OpenCode не грузит плагины"},

    ".sql": {"encoding": "Windows-1251 (ANSI)", "bom": "нет", "note": "Стандартная кодировка SQL-файлов проекта"},

    ".bat": {"encoding": "Windows-1251 (ANSI)", "bom": "нет", "note": "Стандартная кодировка bat-файлов"},

```

```
".md": {"encoding": "UTF-8", "bom": "без BOM", "note": "Документация"},  
}
```

```
RULE_MAP = {  
  
"кодировка": ""
```

Кодировка файлов (из AGENTS.md)

.ps1 - UTF-8 с BOM (PS 5.1 иначе кракозябры)

.json - UTF-8 без BOM (Bug-044: OpenCode не грузит плагины)

.sql - Windows-1251 (ANSI)

.bat - Windows-1251 (ANSI)

.md - UTF-8 без BOM

```
"" ,
```

```
"bash": ""
```

Запуск PowerShell из bash tool

ВСЕГДА: powershell -NoLogo [-File script.ps1]

НИКОГДА: -NoProfile (Bug-046: ломает UTF-8 вывод)

Команды с \$, {}, вложенными кавычками -> только через -File script.ps1

```
"" ,
```

```
"ps5.1": ""
```

Ограничения PS 5.1

- НЕ использовать: ??, ??=, ?. - только if/else

- Функции ДО вызова (PS 5.1 не hoist-ит) - Bug-041

- | в строках парсится как pipe - Bug-026

```
""",  
  
"preflight": ""
```

Preflight перед записью

1. .ps1 -> ДОБАВИТЬ BOM
2. .json/.jsonc -> УБРАТЬ BOM
3. .sql/.bat -> Windows-1251
4. bash с \$ или {} -> ЧЕРЕЗ -File script.ps1
5. oldString != newString перед edit (Bug-039)

```
""",  
  
"task": ""
```

Правило TASK

TASK <имя> [<папка>], по умолчанию: C:\AIS\1 Release

Git_ = оригиналы, Test_ = изменённые, Ready_ = готовые

```
""",  
  
"сохр": ""
```

Правило СОХР

Save-Snapshot.ps1 -- только по команде «СОХР».

Перед правкой opencode.jsonc -- запросить СОХР.

```
""",  
  
"pb": ""
```

Правила PowerBuilder

Редактировать только .sr* в PB_Current.

Event-ы заканчивать 'end event'. Не путать event/function.

call super:: только при выключенном Extend Ancestor Script.

Именованые: uf_ (функции), ue_ (события), i_/li_/ls_/ld_ (переменные).

```
""",  
  
"sql": ""
```

Правила SQL (Sybase ASE 15.5)

STR_REPLACE (не REPLACE), CHARINDEX (2 аргумента), CHAR_LENGTH (не LEN).

SELECT TOP N (не LIMIT), SET ROWCOUNT N (сбрасывать!).

ISNULL (не COALESCE как основной), IDENTITY + @@IDENTITY.

Временные таблицы: #prefix. Ошибки: @@ERROR + GOTO.

```
""",  
  
"vss": ""
```

Правила VSS

Сервер: ren-msksf01, проект \$/SRC125, ss.exe.

Команды: Status, History, Get, Checkout, Checkin, UndoCheckout.

Авторизация: -l-Y -Y"user,pass". Пароль: AES-256 в config.json.

```
""",  
  
"jira": ""
```

Правила Jira

URL: https://mytask.renins.com

Таблица: ||Проект||Библиотека||Наименование||Информация|| (4 пайпа).

NEWPB/OLDPB: Ready_ vs Git_. Формат: "изменены: f1, f2".

```
""",  
  
}
```

```
SOURCE_MAP = {  
  
    "pb": ".opencode/pb-object-rules.mdc",  
  
    "sql": ".opencode/sql-rules.mdc",  
  
    "vss": ".opencode/vss-rules.mdc",  
  
    "jira": ".opencode/jira-table-rules.mdc",  
  
    "project": ".opencode/project-context.mdc",  
  
    "layout": ".opencode/repository-layout.mdc",  
  
    "entity": ".opencode/entity-reference.mdc",  
  
    "glossary": ".opencode/glossary.mdc",  
  
    "preflight": ".opencode/preflight.mdc",  
  
    "russian": ".opencode/russian-language.mdc",  
  
    "pending": "rules/pending-tasks.md",  
  
    "agents": "AGENTS.md",  
  
}
```

```
@mcp.tool()
```

```
def search_bug(keyword: str) -> str:
```

```
    """Поиск по истории багов (rules/changelog.md). Возвращает до 8 релевантных записей."""
```

```
    lines = _get_changelog()
```

```

entries = _extract_bug_entries(lines, keyword)

if not entries:

return f"Баги по ключу '{keyword}' не найдены в changelog.md"

out = [f"Найдено записей: {len(entries)}\n"]

for e in entries:

title = e["title"]

text = e["text"].strip()[:300]

out.append(f"## {title}\n{text}\n---")

return "\n".join(out)

```

```

@mcp.tool()

def get_encoding_rule(file_path: str) -> str:

"""Возвращает требования к кодировке для указанного файла."""

ext = Path(file_path).suffix.lower()

if ext in ENCODING_TABLE:

info = ENCODING_TABLE[ext]

return f"{ext}: {info['encoding']}, BOM: {info['bom']}\n{info['note']}"

return f"Неизвестное расширение '{ext}'. Известны: {', '.join(ENCODING_TABLE.keys())}"

```

```

@mcp.tool()

def get_rule(topic: str) -> str:

"""Возвращает краткую сводку правила по теме.

```

Темы: кодировка, bash, ps5.1, preflight, task, coxp, pb, sql, vss, jira, pending, project."""

```
tl = topic.lower().strip()
```

```
if tl in RULE_MAP:
```

```
    return RULE_MAP[tl]
```

```
available = ", ".join(sorted(RULE_MAP.keys()))
```

```
return f"Тема '{topic}' не найдена. Доступны: {available}"
```

```
@mcp.tool()
```

```
def get_full_rule(source: str) -> str:
```

```
    """Читает полный текст правила из .mdc или .md файла проекта.
```

```
    Источники: pb, sql, vss, jira, project, layout, entity, glossary, preflight, russian, pending, agents."""
```

```
    sl = source.lower().strip()
```

```
    if sl not in SOURCE_MAP:
```

```
        available = ", ".join(sorted(SOURCE_MAP.keys()))
```

```
        return f"Источник '{source}' не найден. Доступны: {available}"
```

```
    rel_path = SOURCE_MAP[sl]
```

```
    full_path = PROD_ROOT / rel_path
```

```
    if not full_path.exists():
```

```
        return f"Файл не найден: {full_path}"
```

```
    try:
```

```
        text = full_path.read_text(encoding="utf-8")
```

```
    except UnicodeDecodeError:
```

```
        text = full_path.read_text(encoding="cp1251")
```

```

lines = text.splitlines()

if len(lines) > 300:

text = "\n".join(lines[:300]) + f"\n\n... [ещё {len(lines)-300} строк, файл обрезан]"

return f"=== {rel_path} ({len(lines)} строк) ===\n\n{text}"

```

```

@mcp.tool()

def get_abbreviation(abbr: str) -> str:

    """Возвращает расшифровку сокращения проекта."""

    abbrs = _get_abbreviations()

    clean = abbr.strip()

    if clean in abbrs:

        return f"{clean} -- {abbrs[clean]}"

    for k, v in abbrs.items():

        if clean.lower() in k.lower() or k.lower() in clean.lower():

            return f"{k} -- {v}"

    total = len(abbrs)

    return f"Сокращение '{clean}' не найдено. Всего: {total}."

```

```

@mcp.tool()

def preflight_check(file_path: str, operation: str = "write") -> str:

    """Preflight-проверка перед записью файла."""

    ext = Path(file_path).suffix.lower()

```

```

lines: list[str] = []

if ext in ENCODING_TABLE:

    info = ENCODING_TABLE[ext]

    lines.append(f"[КОДИРОВКА] {info['encoding']}, BOM: {info['bom']}")

    lines.append(f" !! {info['note']}")

if ext == ".ps1":

    lines.append("[PS 5.1] Проверить:")

    lines.append(" - Нет ??, ??=, ?.")

    lines.append(" - Функции ДО вызова (Bug-041)")

if ext in (".ps1", ".json", ".jsonc"):

    if "bash" in operation.lower() or "cmd" in operation.lower():

        lines.append("[BASH] powershell -NoLogo (без -NoProfile!)")

    if operation.lower() == "edit":

        lines.append("[EDIT] oldString != newString ? (Bug-039)")

if not lines:

    lines.append(f"Нет специальных правил для '{ext}'.")

return f"=== Preflight: {Path(file_path).name} ({operation}) ===\n" + "\n".join(lines)


if __name__ == "__main__":

    mcp.run()

--- КОНЕЦ ФАЙЛА server.py ---

```

Шаг 3: Проверить

1. Проверить синтаксис: `python -c "import py_compile; py_compile.compile(r'C:\AIS\AI\rules-mcp\rules_mcp\server.py', doraise=True)"`
2. Перезапустить OpenCode
3. Вызвать `get_full_rule("sql")` — должен вернуть полный текст `sql-rules.mdc`